INTRODUCTION TO A.I – END TERM

Nguyen Dong Quan
Student ID: 523h0171
Faculty of IT, TDTU
HCMC, VN
523h0171@student.tdtu.edu.vn

Pham Quoc Hung
Student ID: 523h0135
Faculty of IT, TDTU
HCMC, VN
523h0135@student.tdtu.edu.vn

Dinh Bui Khanh Huy
Student ID: 523h0136
Faculty of IT, TDTU
HCMC, VN
523h0136@student.tdtu.edu.vn

Ths. Nguyen Thanh An

***Abstract*—Summarize**

- Requirements:This project consists of five tasks, each representing a different application of Artificial Intelligence:

- **Simulated Annealing for Optimization on 3D surfaces**
- **Heuristic Alpha-Beta Search for Strategic Gameplay in a 9x9 Tic-Tac-Toe Game**
- **Solving Constraint Satisfaction Problems Using Propositional Logic and SAT solvers**
- **Classifying Student Performance Using a Nave Bayesian Classifier.**

- Approaches: each task using suitable AI techniques

- **Simulated Annealing Search with a custom temperature schedule to find the global maximum on a given surface.**
- **Alpha-Beta pruning with heuristic evaluation and depth limitation for the AI in Tic-Tac-Toe.**
- **SAT encoding and clause generation for grid-based logic constraints, solved via PySAT's Glucose3**
- **Discretization and manual probability calculations for the Naïve Bayesian Classifier.**

- Experimental results: Experiments showed that all tasks performed correctly and as expected. The 3D plot visualized search paths, the AI in Tic-Tac-Toe made intelligent decisions, the CSP solver produced valid outputs, and the classifier yielded good accuracy.

- Levels of completion: 87%

I. Task 1: Simulated Annealing Search on 3D surfaces

Theory :

-Combine hill climbing with a random walk in some way that yields both efficiency and completeness

-Shake hard (i.e., at a high temperature) and then gradually reduce the intensity of shaking (i.e., lower the temperature)

-Pseudo:

```
function SIMULATED-ANNEALING(problem, schedule)
returns a solution state
inputs: problem, a problem
        schedule, a mapping from time to "temperature"
    current ← MAKE-NODE(problem.INITIAL-STATE)
    for t = 1 to ∞ do
        T ← schedule(t)
        if T = 0 then return current
        next ← a randomly selected successor of curren
        ΔE ← next.VALUE – current.VALUE
        if ΔE > 0 then current ← next
        else current ← next only with probability e^(ΔE/T)
```

Approach :

-This task aims to find the maximum of a complex 2D mathematical function using Simulated Annealing (SA). The target function:

$$f(x,y) = \sin\left(\frac{x}{8}\right) + \cos\left(\frac{y}{4}\right) - \sin\left(\frac{xy}{16}\right) + \cos\left(\frac{x^2}{16}\right) + \sin\left(\frac{y^2}{8}\right) \quad (1)$$

is defined using sympy and numpy.

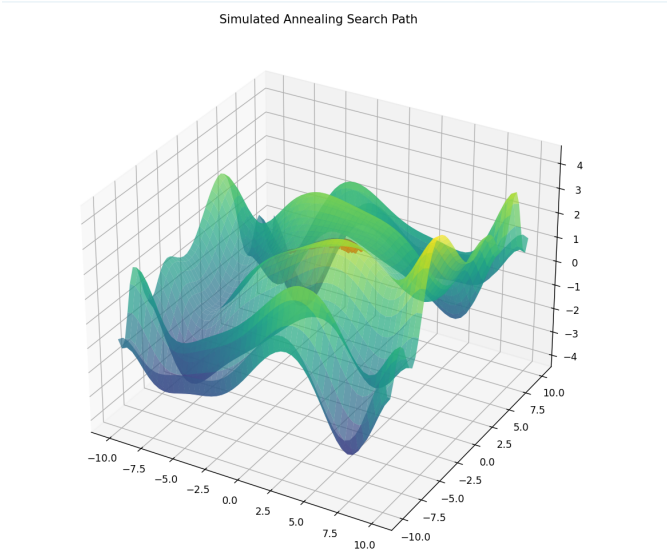
-Simulated Annealing starts from a random point, explores neighbors in four directions, and uses a cooling schedule $T(t) = \frac{1}{1+0.01t}$ to control acceptance of worse solutions.

Result :

- Best solution

```
Running simulated annealing...
Best solution found: Solution(x=3.3379, y=-3.5343, value=3.4791)
```

- Path:



II. Task 2: 9x9 Tic-Tac-Toe with Heuristic Alpha-Beta Search

Theory :

-Alpha-beta pruning: Minimax algorithm but prunning away branches that cannot possibly influence the final decision

-Pseudo:

```
function ALPHA-BETA-SEARCH(state) returns an action
    v ← MAX-VALUE(state,-∞,+∞)
    return the action in ACTIONS(state) with value v

function MAX-VALUE(state,α,β) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← -∞
    for each a in ACTIONS(state) do
        v ← MAX(v, MIN-VALUE(RESULT(s,a),α,β))
        if v ≥ β then return v
        α ← MAX(α, v)
    return v

function MIN-VALUE(state,α,β) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← +∞
    for each a in ACTIONS(state) do
        v ← MIN(v, MAX-VALUE(RESULT(s,a) ,α,β))
        if v ≤ α then return v
        β ← MIN(β, v)
    return v
```

Approach :

-State Representation: Use a State class to represent the 9x9 board, provide valid moves, check terminal states, and calculate utility.

-Problem Definition: The Tic-Tac-Toe Problem manages the game rules, transitions, legal moves, and terminal checks.

-Evaluation Function: Pattern-based heuristic that rewards AI lines, penalizing opponent threats, and favoring central moves.

-Search Algorithm: Minimax with alpha-beta pruning, limited search depth, and heuristic evaluation at leaf nodes for efficient decision making.

Result:

Heuristic Score Example:

Line	Type	Score
`. o o o .`	Attack	10,000
`x x x .`	Defense	-10,000
`o . . .`	Potential	50

Step	Player	Move	Symbol	Score
1	X	(0, 0)	X	-150
2	O	(2, 2)	O	401
3	X	(4, 0)	X	51
4	O	(2, 3)	O	1903
5	X	(3, 4)	X	1103
6	O	(2, 4)	O	21056
7	X	(4, 3)	X	18656
8	O	(2, 5)	O	120008

III. Task 3: Constraint Satisfaction Problems with Propositional Logic

Theory :

-A constraint satisfaction problem (CSP) uses a factored representation for each state.

- State = a set of variables and each of which has a value
- Solution = each variable has a value that satisfies all constraints on that variable

-Take advantage of the structure of states

-General-purpose rather than problem-specific heuristics

- Identify combinations of variable-value that violate the constraints → eliminate large portions of the search space all at once
- Solutions to complex problems

-A CSP consists of the following three components

- X = $\{X_1, .., X_n\}$: a set of variables
- D = $\{D_1, .., D_n\}$: a set of domains, one for each variable.
- C a set of constraints that state allowable combinations of values.

-Each C consists of a pair <scope, rel>

- scope: a tuple of variables that participate in the constraint
- A relation rel defines the values that participated variables can take

Approach :

-Problem Formulation:

- Variables: Each cell in the 10×10 grid.
- Domain: Digits (e.g., 0–9), possibly restricted.
- Constraints: All-different rows/columns, sum/relational constraints.

-CSP Techniques Used:

- Backtracking Search with:
- Forward Checking

- MRV, Degree Heuristic, LCV for efficient variable/value selection.

Result:

-Input:

```
input.txt ×
1 . 2 3 . . 0 . . . .
2 . . . . 3 . 2 . . 6
3 . . 5 . 5 3 . 5 7 4
4 . 4 . 5 . 5 . 6 . 3
5 . . 4 . 5 . 6 . . 3
6 . . . 2 . 5 . . . .
7 4 . 1 . . . 1 1 . .
8 4 . 1 . . . 1 . 4 .
9 . . . . 6 . . . . 4
10 . 4 4 . . . . 4 . .
```

-Output:

```
Solution found:
Green = True cells, Red = False cells
. 2 3 . . 0 . . . .
. . . . 3 . 2 . . 6
. . 5 . 5 3 . 5 7 4
. 4 . 5 . 5 . 6 . 3
. . 4 . 5 . 6 . . 3
. . . 2 . 5 . . . .
4 . 1 . . . 1 1 . .
4 . 1 . . . 1 . 4 .
. . . . 6 . . . . 4
. 4 4 . . . . 4 . .
```

IV. Task 4: Na¨ve Bayesian Classifier

Theory :

-The Naive Bayes classifier is a simple probabilistic classification model based on Bayes' Theorem. It is particularly effective for high-dimensional data and text classification problems.

-Given a data instance $\mathbf{X} = (x_1, x_2, \ldots, x_n)$ and a set of m possible classes $C_1, C_2, \ldots, C_m$, the classifier estimates the most probable class label $\hat{C}$ using Bayes' Theorem:

$$\hat{C} = \arg\max_{C_i} P(C_i \mid \mathbf{X}) = \arg\max_{C_i} \frac{P(\mathbf{X} \mid C_i) \cdot P(C_i)}{P(\mathbf{X})} \quad (2)$$

Since $P(\mathbf{X})$ is constant across all classes, it can be omitted during maximization:

$$\hat{C} = \arg\max_{C_i} P(C_i) \cdot P(\mathbf{X} \mid C_i) \quad (3)$$

-The Naive Bayes assumption assumes that the features $x_1, x_2, \ldots, x_n$ are conditionally independent given the class C_i. Therefore, the likelihood can be expressed as:

$$P(\mathbf{X} \mid C_i) = \prod_{k=1}^{n} P(x_k \mid C_i) \quad (4)$$

Substituting into the classification rule yields the final Naive Bayes decision function:

$$\hat{C} = \arg\max_{C_i} P(C_i) \cdot \prod_{k=1}^{n} P(x_k \mid C_i) \quad (5)$$

-Note: $\hat{C}$ (read as "C hat") denotes the predicted class label—that is, the class with the highest posterior probability given the observed features.

-The Naive Bayes classifier is computationally efficient and performs well in many real-world applications, including spam filtering, sentiment analysis, and document categorization. However, its strong independence assumption may limit performance when features are highly correlated.

Approach :

-Problem Setup: Use a labeled dataset (task4_data.csv) to predict class labels of new data.

-Naïve Bayes Classifier:

- Assumes feature independence.
- Applies Bayes' Theorem to compute class probabilities.
- Chooses the class with the highest posterior.

-Types Handled:

- Gaussian (numerical features).
- Multinomial/Bernoulli (categorical, if encoded).

-Workflow:

- Preprocess data (handle missing values, encode features).
- Train using priors and likelihoods.
- Evaluate on test set.

Result :

-Input: data.csv

-Output:

```
Discretizing data with 3 bins (equal-width bins):

Q1 bins:
  bin_0: [0.00, 2.33)
  bin_1: [2.33, 4.67)
  bin_2: [4.67, 7.01)
Q2 bins:
  bin_0: [0.00, 3.33)
  bin_1: [3.33, 6.67)
  bin_2: [6.67, 10.01)
Q3 bins:
  bin_0: [0.00, 3.33)
  bin_1: [3.33, 6.67)
  bin_2: [6.67, 10.01)
Q4 bins:
  bin_0: [0.00, 3.33)
  bin_1: [3.33, 6.67)
  bin_2: [6.67, 10.01)
Q5 bins:
  bin_0: [0.00, 3.00)
  bin_1: [3.00, 6.00)
  bin_2: [6.00, 9.01)
Q6 bins:
  bin_0: [0.00, 3.33)
  bin_1: [3.33, 6.67)
  bin_2: [6.67, 10.01)
Q7 bins:
  bin_0: [0.00, 2.67)
  bin_1: [2.67, 5.33)
  bin_2: [5.33, 8.01)
Q8 bins:
  bin_0: [0.00, 3.10)
  bin_1: [3.10, 6.20)
  bin_2: [6.20, 9.31)
Q9 bins:
  bin_0: [0.00, 3.33)
  bin_1: [3.33, 6.67)
  bin_2: [6.67, 10.01)
```

```
Classification accuracy: 0.7603
```

V. Contributions

Name	Task 1	Task 2	Task 3	Task 4	Report
Nguyen Dong Quan	10%	70%	10%	20%	70%
Pham Quoc Hung	10%	15%	70%	65%	5%
Dinh Bui Khanh Huy	80%	15%	20%	15%	25%

VI. Self-evaluation

Task	Score
Task 1	1.5
Task 2	2.5
Task 3	2.5
Task 4	0.5
Report	0.5

VII. Conclusion

Summarize

- **Requirements**:This project consists of five tasks, each representing a different application of Artificial Intelligence:

- Simulated Annealing for Optimization on 3D surfaces
- Heuristic Alpha-Beta Search for Strategic Gameplay in a 9x9 Tic-Tac-Toe Game
- Solving Constraint Satisfaction Problems Using Propositional Logic and SAT solvers
- Classifying Student Performance Using a Nave Bayesian Classifier.

- **Approaches**: each task using suitable AI techniques

- Simulated Annealing Search with a custom temperature schedule to find the global maximum on a given surface.
- Alpha-Beta pruning with heuristic evaluation and depth limitation for the AI in Tic-Tac-Toe.
- SAT encoding and clause generation for grid-based logic constraints, solved via PySAT's Glucose3
- Discretization and manual probability calculations for the Naïve Bayesian Classifier.

- **Experimental results**: Experiments showed that all tasks performed correctly and as expected. The 3D plot visualized search paths, the AI in Tic-Tac-Toe made intelligent decisions, the CSP solver produced valid outputs, and the classifier yielded good accuracy.

- **Levels of completion**: 87%

References

[1] Stuart Russell and Peter Norvig. 2009. Artificial Intelligence: A Modern Approach (3rd ed.). Prentice Hall Press, Upper Saddle River, NJ, USA.

[2] Lê Hoài Bắc, Tô Hoài Việt. 2014. Giáo trình Cơ sở Trí tuệ nhân tạo. Khoa Công nghệ Thông tin. Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM.

[3] Nguyễn Ngọc Thảo, Nguyễn Hải Minh. 2020. Bài giảng Cơ sở Trí tuệ Nhân tạo. Khoa Công nghệ Thông tin. Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM.